

# ELEMENTOS MAQUETACIÓN GRID



1.

¿GRID?

“

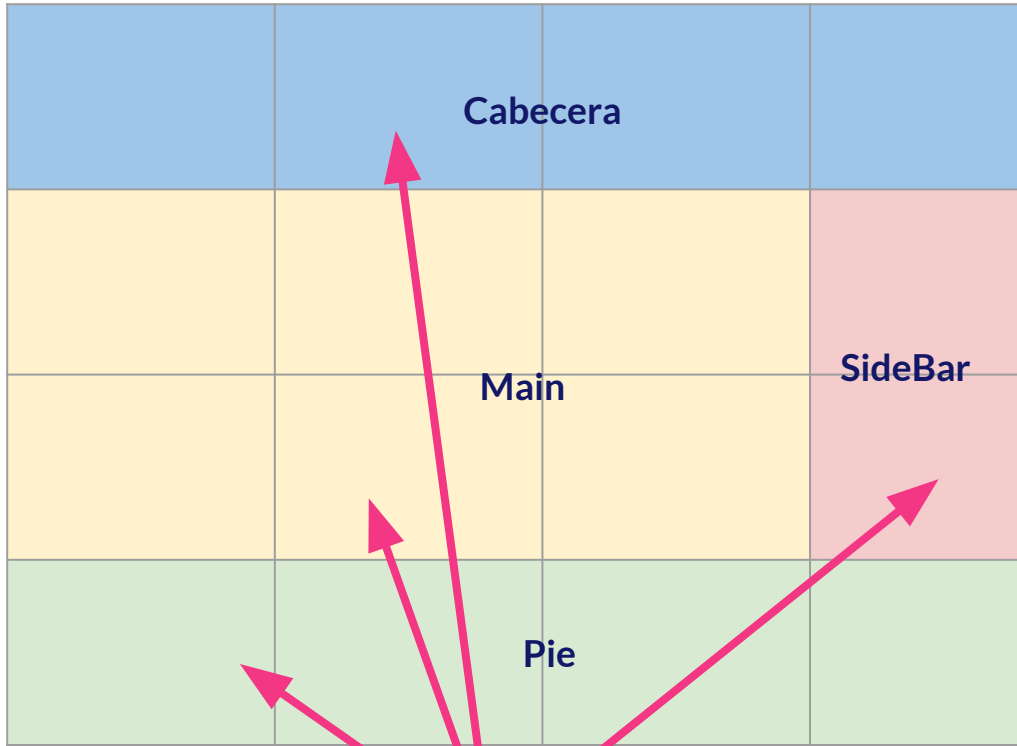
La **idea principal** de la maquetación con elemento **GRID** es que vamos a tener un elemento (una etiqueta) que va a poder **controlar** propiedades de los elementos y establecer **estructuras complejas** para distribuirlos.

# 2.

## ELEMENTOS

# ELEMENTOS DE MAQUETACIÓN GRID

- ▶ El contenedor **GRID** (**display:grid** o **display:inline-grid**)
- ▶ Los **elementos grid** que están dentro, cuyas propiedades modificaremos y que colocaremos atendiendo a una estructura compleja.



Elementos del Grid

Contenedor con  
*display: grid / inline-grid*

3.

**PROPIEDADES**

## PROPIEDADES CONTENEDOR GRID

- ▶ Estructura en filas y columnas y la separación entre ellas.
- ▶ Definir áreas del GRID con nombre
- ▶ Las alineación horizontal y vertical de los elementos del GRID y del propio GRID

Es decir propiedades que usamos para **maquetar** y de una forma más fácil a la forma tradicional.

# 4.

## ESTRUCTURA DEL GRID

## PARA DEFINIR LA ESTRUCTURA

- ▶ **grid-template-columns** : Número de columnas del grid y tamaño de las mismas
- ▶ **grid-template-rows**: Número de filas del grid y tamaño de las mismas.
- ▶ **grid-row-gap**: Separación entre las filas.
- ▶ **grid-column-gap**: Separación entre las columnas.

# COLUMNAS

- ▶ **grid-template-columns:** Debemos de poner tantos valores de anchura como columnas quiero que tenga el GRID.

*grid-template-columns: 20% 50% 30%*

*grid-template-columns: 100px auto 100px 100px*

*grid-template-columns: auto auto auto auto auto*

*grid-template-columns : [id] 100px [nombre] 300px  
[apellidos] auto*

## FILAS

- ▶ **grid-template-rows:** Debemos de poner tantos valores de altura como filas quiero que tenga el GRID.

*grid-template-rows: 20% 50% 30%*

*grid-template-rows: 100px auto 100px 100px*

*grid-template-rows: auto auto auto auto auto*

*grid-template-rows : [uno] 100px [dos] 300px [tres] auto*

## REPETIR VALORES Y UNIDAD FR

*grid-template-column: repeat(3, 20% [col-start]) auto*

*grid-template-column: 2fr 100px 1fr 2fr*

5.

## ALINEACIÓN HORIZONTAL

“

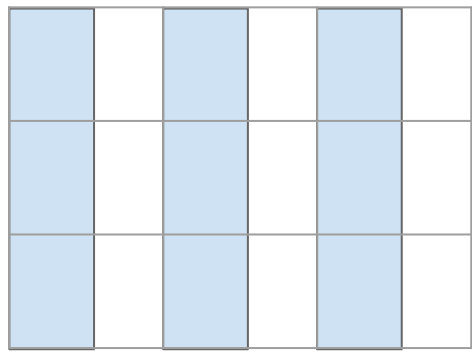
Para **alinear horizontalmente** el contenido que hay dentro de cada celda usaré la propiedad CSS

**justify-items**

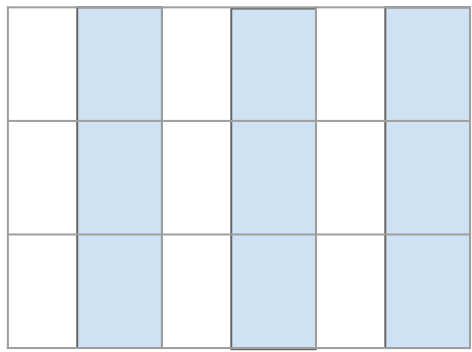
# VALORES PARA JUSTIFY-ITEMS

- ▶ start
- ▶ end
- ▶ center
- ▶ stretch (por defecto)

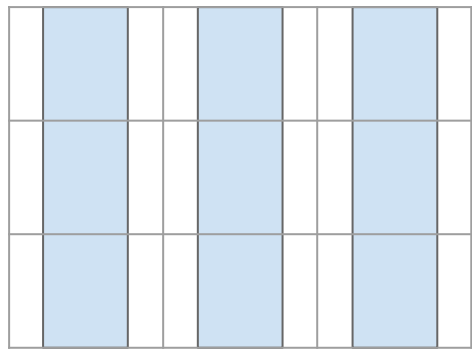
start



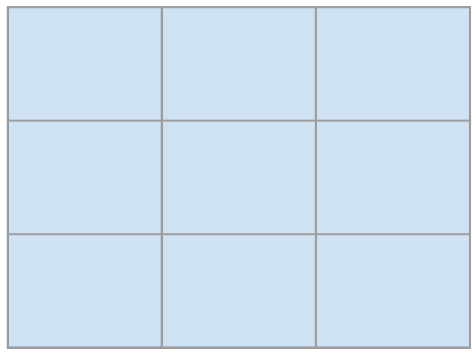
end



center



stretch



# 6.

## ALINEACIÓN VERTICAL

“

Para **alinear verticalmente** el contenido que hay dentro de cada celda usaré la propiedad CSS **align-items**

## VALORES PARA **ALIGN-ITEMS**

- ▶ start
- ▶ end
- ▶ center
- ▶ stretch (por defecto)

end

|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |

# stretch

|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |

“

Puedo juntar las dos alineaciones anteriores en la propiedad **place-items**

*place-items: start end*

Vertical   Horizontal

Two red arrows point from the words 'Vertical' and 'Horizontal' below to the words 'start' and 'end' in the 'place-items' property definition above. The arrow from 'Vertical' points to 'start', and the arrow from 'Horizontal' points to 'end'.

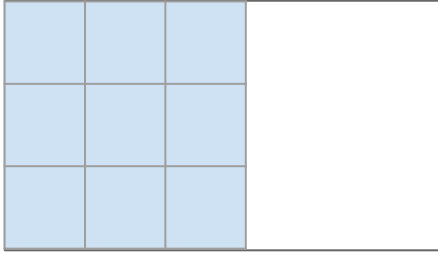
# 7.

## DISTRIBUCIÓN DENTRO DEL CONTENEDOR

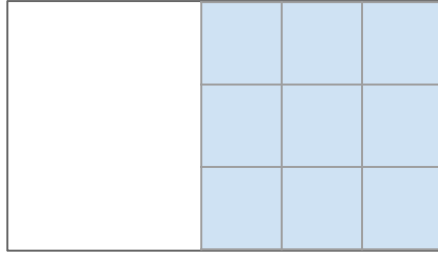
“

En el caso de que el GRID **no ocupe todo el contenedor GRID** podemos distribuirlo con las propiedades CSS **justify-content** (horizontal) y **align-content** (vertical)

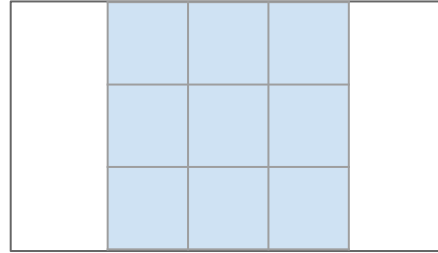
start



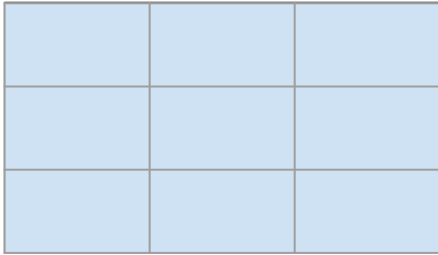
end



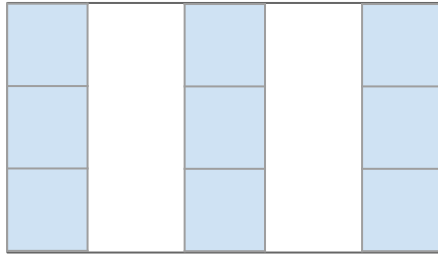
center



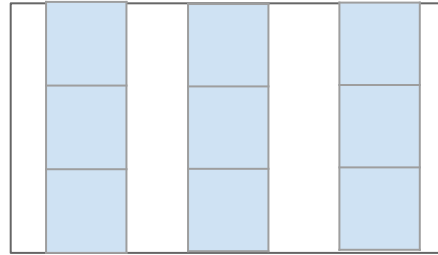
stretch



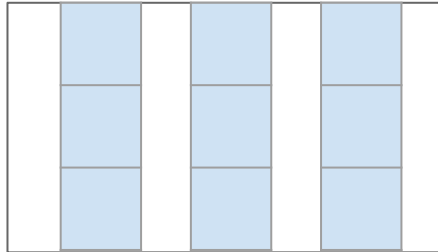
space-between



space-around

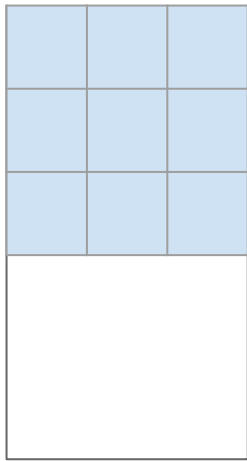


JUSTIFY-CONTENT

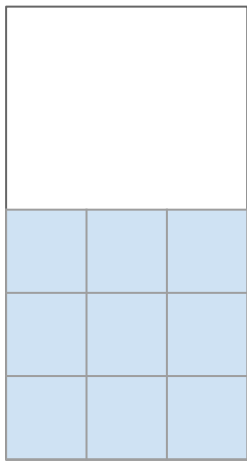


space-evenly

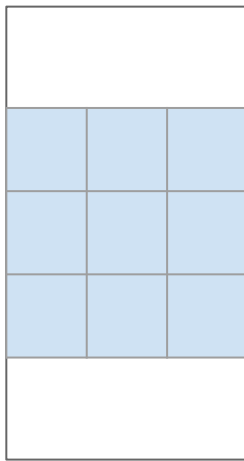
start



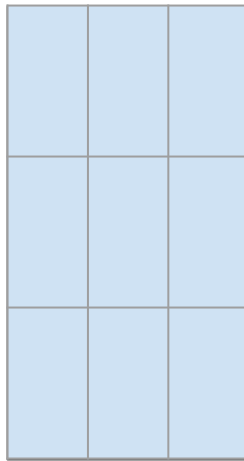
end



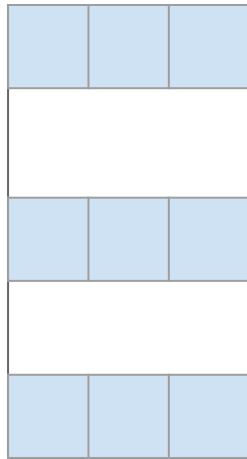
center



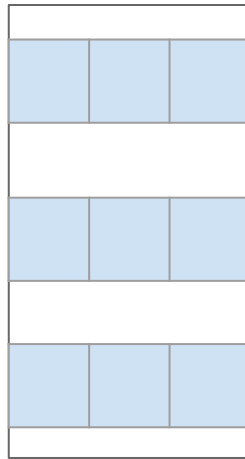
stretch



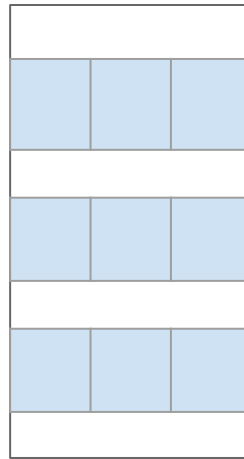
ALIGN-CONTENT



space-between



space-around



space-evenly

“

Puedo juntar las dos distribuciones anteriores en la propiedad **place-content**

*place-content: start end*

Vertical   Horizontal

A diagram illustrating the mapping of the 'place-content' property values to the 'start' and 'end' sub-properties. Two red arrows point upwards from the words 'Vertical' and 'Horizontal' to the 'start' and 'end' keywords in the 'place-content: start end' line above. The arrow from 'Vertical' points to 'start', and the arrow from 'Horizontal' points to 'end'.



# 8.

## ELEMENTOS GRID

¿qué era eso?

“

Los **elementos GRID** son los **hijos** directos (dentro del árbol DOM) del **elemento** con la propiedad CSS **display: grid**

## PROPIEDADES **ELEMENTOS** GRID

- ▶ **Área del GRID** que va a ocupar.
- ▶ **Alineación horizontal** de manera individual.
- ▶ **Alineación vertical** de manera individual.
- ▶ **Colocación implícita** si no especificamos nada.

**No les afecta:** float, display:inline-block, display-table-cell, vertical-align o column-\*

9.

**AREA A OCUPAR**

¿dónde los  
coloco?

## PARA ESPECIFICAR EL AREA

- ▶ `grid-column-start`
- ▶ `grid-column-end`
- ▶ `grid-row-start`
- ▶ `grid-row-end`

|          |   |   |   |   |   |
|----------|---|---|---|---|---|
|          | 1 | 2 | 3 | 4 | 5 |
| [uno]    |   |   |   |   |   |
| [dos]    |   |   |   |   |   |
| [tres]   |   |   |   |   |   |
| [cuatro] |   |   |   |   |   |



grid-column-start: 1;  
grid-column-end: 2;  
grid-row-start: uno;  
grid-row-end: cuatro;



grid-column-start: 2;  
grid-column-end: span 3;  
grid-row-start: uno;  
grid-row-end: dos;



grid-column-start: 2;  
grid-column-end: 5;  
grid-row-start: dos;  
grid-row-end: span 2;

“

Podemos juntar estas propiedades:

*grid-column: start / end*

*grid-row: start / end*

*grid: row-start column-start row-end column-end*

# 10.

## ALINEACIÓN HORIZONTAL

## PROPIEDAD **JUSTIFY-SELF**

- ▶ **start**
- ▶ **end**
- ▶ **center**
- ▶ **stretch** (por defecto)

Los valores son los mismos que usábamos en la propiedad **justify-items** del contenedor **grid**.

# 11.

## ALINEACIÓN VERTICAL

## PROPIEDAD **ALIGN-SELF**

- ▶ **start**
- ▶ **end**
- ▶ **center**
- ▶ **stretch** (por defecto)

Los valores son los mismos que usábamos en la propiedad **align-items** del **contenedor grid**.

# 12.

## COLOCACIÓN “IMPLÍCITA”

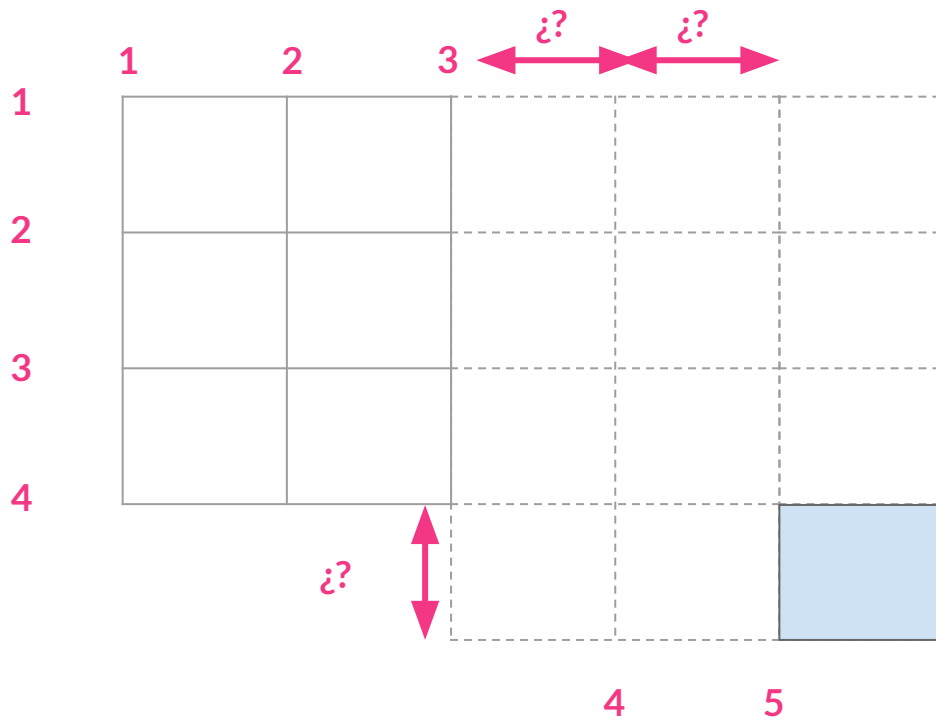
“

La **colocación implícita** de los **elementos GRID** es lo que sucede cuando colocamos elementos GRID **fuera** de la estructura que se ha definido en el **contenedor GRID** o cuando **no les damos posición**.

# COLOCACIÓN IMPLÍCITA

- ▶ `grid-auto-columns`
- ▶ `grid-auto-rows`
- ▶ `grid-auto-flow`

Son propiedades que damos al **CONTENEDOR**.



```
{
  grid-column: 5/6;
  grid-row: 4/5;
}
```

```
.container {
  grid-auto-columns: 50px;
  grid-auto-rows: 50px;
}
```

} (por defecto 0)

“

Si tengo elementos **sin definir una colocación** en el contenedor GRID la propiedad del contenedor ***grid-auto-flow*** define cómo se van a colocar esos elementos.

## VALORES PARA **GRID-AUTO-FLOW**

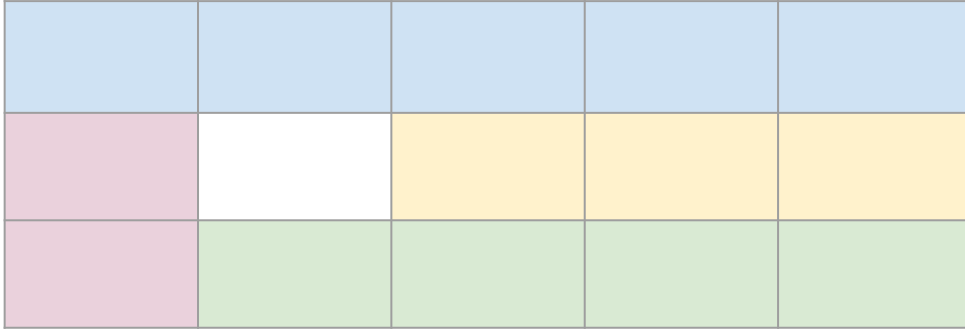
- ▶ **row** : rellena la filas primero (opción por defecto).
- ▶ **column**: rellena las columnas primero.
- ▶ **dense**: intenta rellenar primero los huecos si vienen elementos más pequeños. Puede cambiar el orden de los elementos así que hay que tener cuidado.

# 13.

## MAQUETAR CON ÁREAS

“

Si no queremos especificar el tamaño para cada elemento del GRID podemos darles nombre a su propiedad **grid-area** y usar esos nombres en la propiedad **grid-template-area**



```
.container {
  grid-template-columns: repeat(5, 20%);
  grid-template-rows: repeat(3, 100px);
  grid-template-areas:
    "cap cap cap cap cap"
    "menu . main main main"
    "menu peu peu peu peu";
}
```

Hueco



```
{
  grid-area: main;
}
```



```
{
  grid-area: peu
;}
```



```
{
  grid-area: cap;
}
```



```
{
  grid-area: menu;
}
```

“

Podemos juntar todas en una propiedad

***grid-template:***

Y todas las del contenedor en:

**grid:**

# 14.

## ¿FLEX O GRID?

“

**Independientemente** de qué sistema utilicemos maquetar usando **FLEX** o usando **GRID** es **mucho más fácil** que **maquetar** usando sistemas tradicionales basados únicamente en float, position, tablas etc.

# 15.

**SOBRE FLEX**

# CARACTERÍSTICAS DE FLEX

## Ventajas

- ▶ Una sola dirección
- ▶ Controló fácilmente la alineación de los elementos flexibles.

## Desventajas

- ▶ Para el mismo layout que GRID necesito una estructura HTML más compleja

# 16.

## SOBRE GRID

# CARACTERÍSTICAS DE GRID

## Ventajas

- ▶ Dos dimensiones
- ▶ Puedo definir muy fácilmente la estructura general
- ▶ Es muy fácil definir la separación (gap) entre la celdas

## Desventajas

- ▶ La alineación “dentro” de las celdas

# 17.

## CONCLUSIÓN

## POSIBLE CONCLUSIONES

- ▶ Hay cosas que solo puedo hacer con Flex.
- ▶ Hay cosas que solo puedo hacer con Grid.

# USA

# LAS DOS

Nada impide que un **área** de un **contenedor GRID** sea a su vez un **contenedor FLEX**.

